



An-Najah National University
Computer Engineering Department
Distributed Operating Systems

Bazar Bookstore Project
Part 2 Report

Replication, Caching, and Consistency

Student Names:	1) Hoor Yaish 2) Deema Hazeem
Course	Distributed Operating Systems
Project	Bazar Bookstore - Part 2
Implementation	Node.js, Express.js, SQLite, Docker Compose

Bazar Bookstore - Part 2 Report

1. Introduction

This report explains the second part of the Bazar Bookstore project. Part 2 extends the original Lab 1 bookstore system by adding distributed-system features such as replication, caching, cache consistency, load balancing, and performance measurements. The system was implemented using Node.js with Express.js, SQLite for persistent storage, Axios for REST communication between services, and Docker Compose for deployment.

In this part, the backend services were replicated. The final system contains one frontend service, two catalog replicas, and two order replicas. The frontend receives client requests, performs caching for read operations, and distributes requests among the replicas using round-robin load balancing.

2. Project Structure

The project is divided into independent microservice folders. Each service has its own app.js file, package files, Dockerfile, and database files. The experiments folder contains the performance measurement script, while the docs folder contains the documentation and output files.

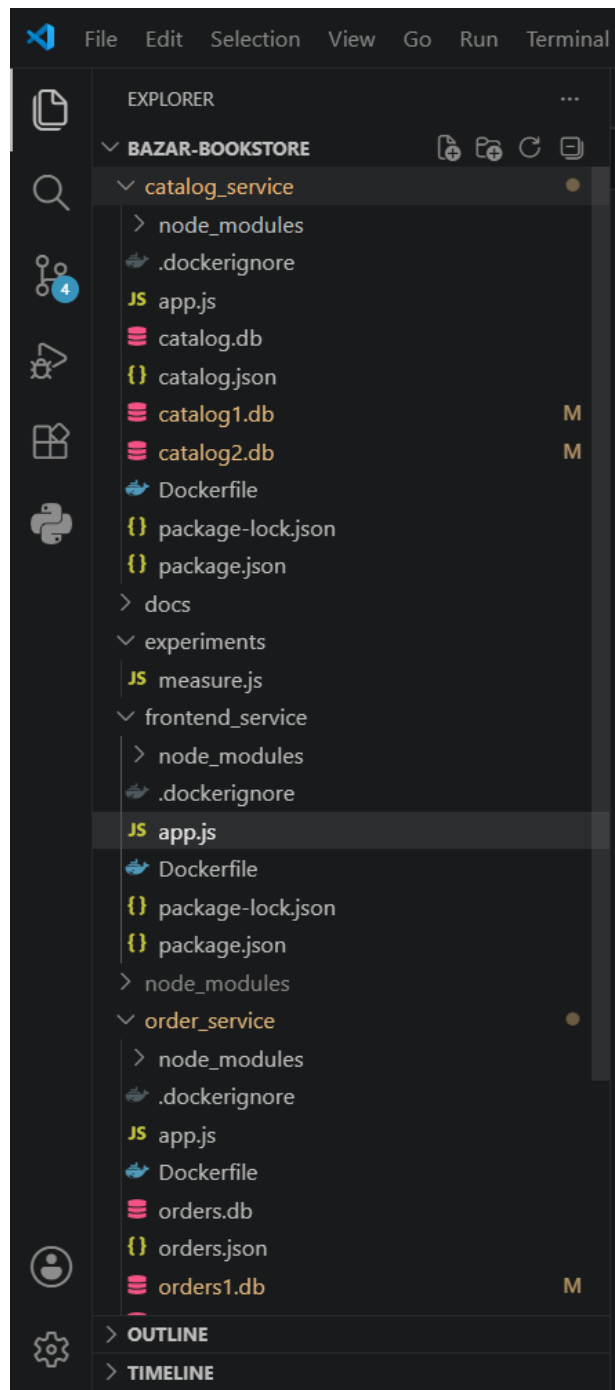


Figure 1: Project hierarchy showing service folders, documentation folder, and experiment script.

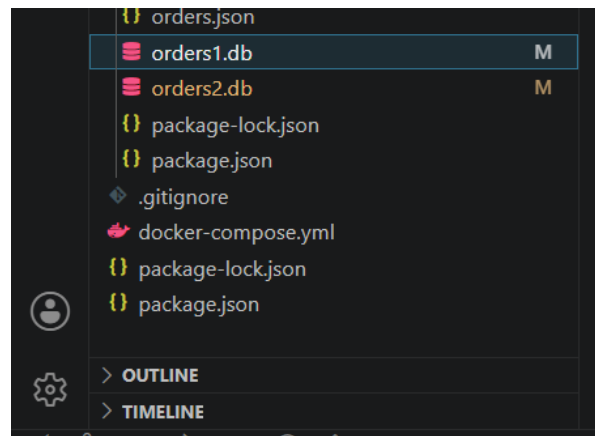


Figure 2: Project hierarchy continuation showing Docker Compose and order replica databases.

3. Lab 2 System Architecture

The final Lab 2 architecture contains five running services: one frontend service, two catalog replicas, and two order replicas. The frontend is the single client-facing service. It forwards read requests to catalog replicas and purchase requests to order replicas.

- Frontend service: receives user requests, checks cache, and load-balances backend requests.
- Catalog replica 1 and catalog replica 2: store book data and synchronize catalog updates.
- Order replica 1 and order replica 2: process purchases and synchronize order records.
- Docker network: allows containers to communicate using service names such as catalog1, catalog2, order1, and order2.

Service	Host Port	Container Port	Main Responsibility
frontend	5000	5000	Client entry point, cache, and load balancer
catalog1	5001	5001	First catalog replica
catalog2	5003	5001	Second catalog replica
order1	5002	5002	First order replica
order2	5004	5002	Second order replica

4. Docker Deployment

Docker Compose was used to run the complete distributed system. Each service runs in its own container. The catalog and order services are each replicated into two containers. The containers are connected to the same Docker bridge network, which allows them to communicate through internal service names.

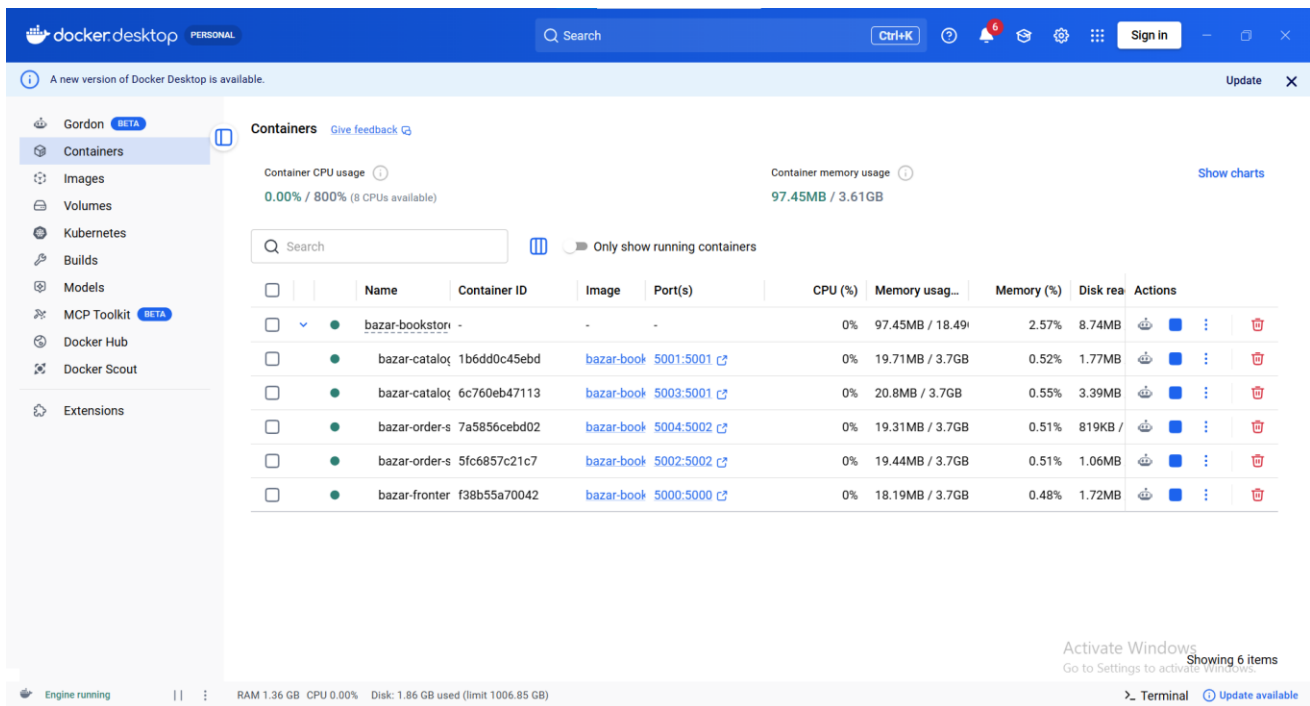


Figure 3: Docker Desktop showing the five Lab 2 containers running successfully.

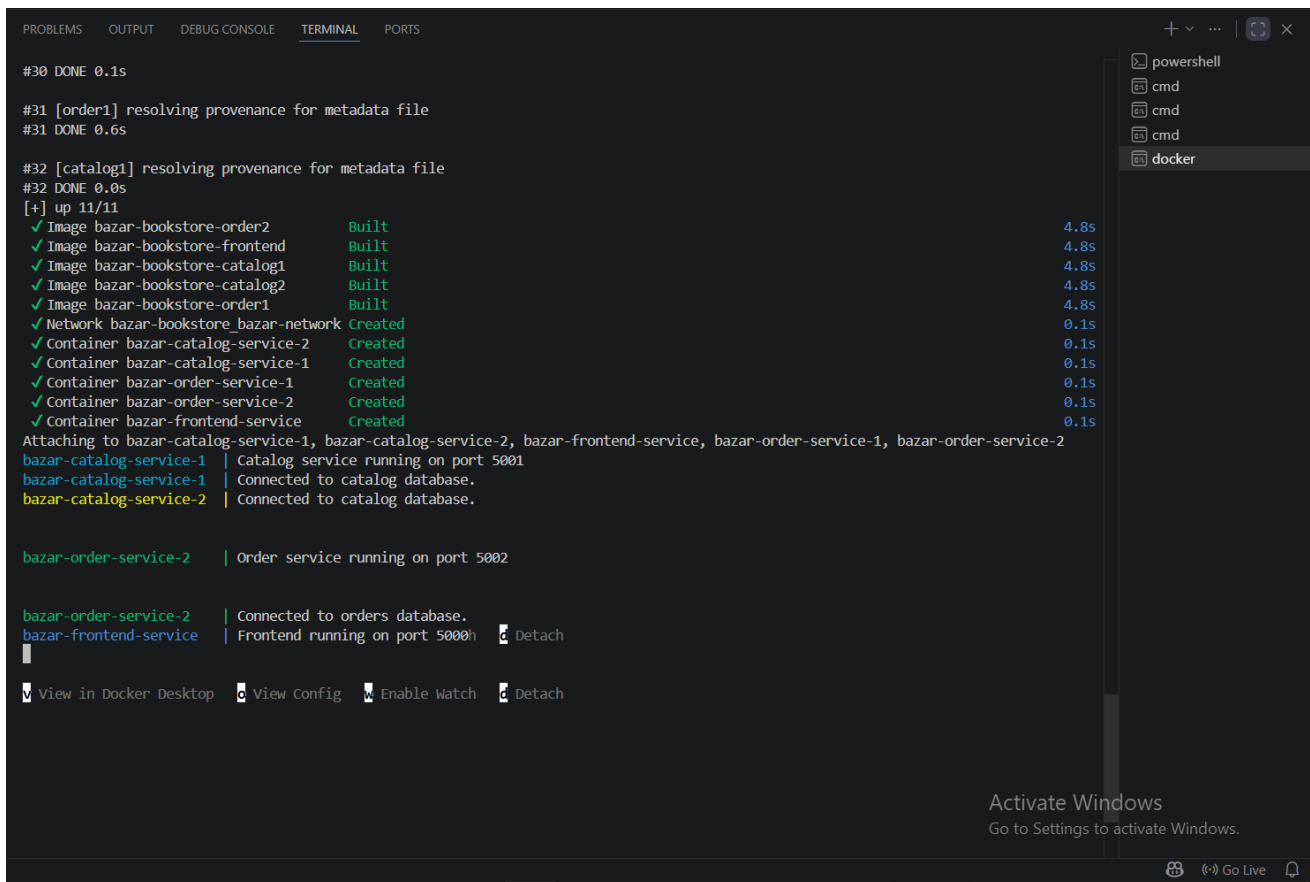


Figure 4: Terminal output showing the frontend, catalog replicas, and order replicas starting successfully.

5. New Books Added in Lab 2

Lab 2 required adding three new books to the catalog. These books were added to the catalog database initialization code and can be accessed through the frontend info endpoint.

ID	Title	Topic	Price
5	How to finish Project 3 on time	undergraduate school	45
6	Why theory classes are so hard	undergraduate school	35
7	Spring in the Pioneer Valley	undergraduate school	25

The following request tests one of the new books:

```
GET http://localhost:5000/info/5
```

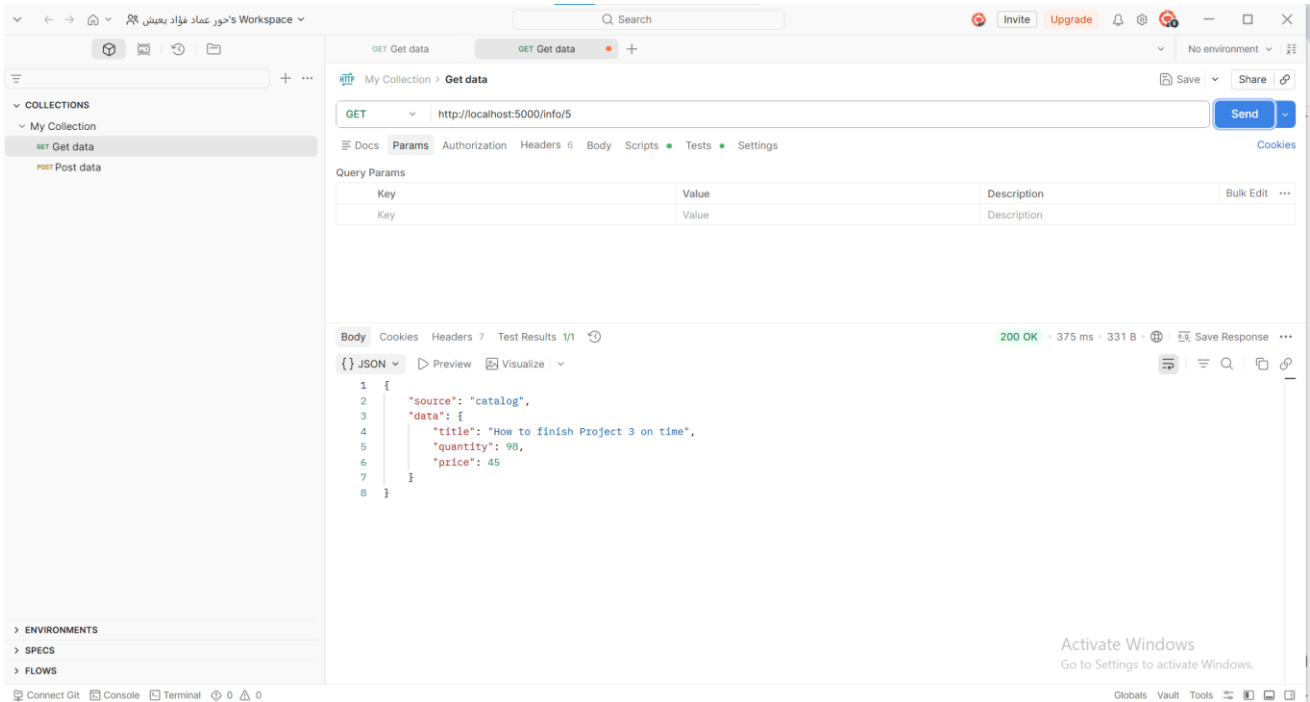


Figure 5: API response showing one of the three new Lab 2 books.

6. Frontend In-Memory Cache

An in-memory cache was implemented inside the frontend service. It stores recent info request results using keys such as `info:5`. The cache is used only for read operations, especially `/info/:id`. Purchase requests are not served from cache because they modify the stock quantity.

Before testing the cache, the cache was cleared manually through the DELETE `/cache` endpoint:

```
DELETE http://localhost:5000/cache
```

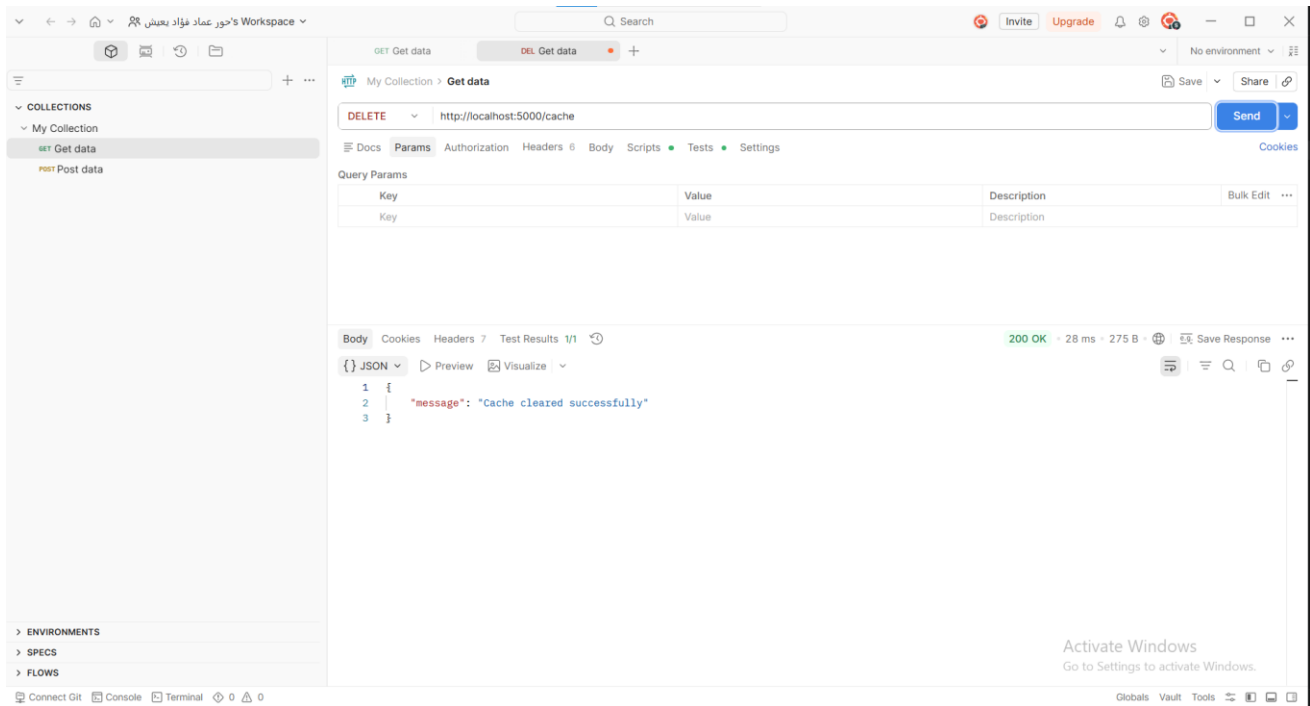


Figure 6: Cache cleared successfully before testing cache behavior.

7. Cache Miss and Cache Hit

When the frontend receives an info request, it first checks the cache. If the item is not found, the request becomes a **cache miss**, and the frontend forwards the request to a catalog replica. The result is then stored in the cache. If the same item is requested again, the frontend returns it directly from memory as a **cache hit**.

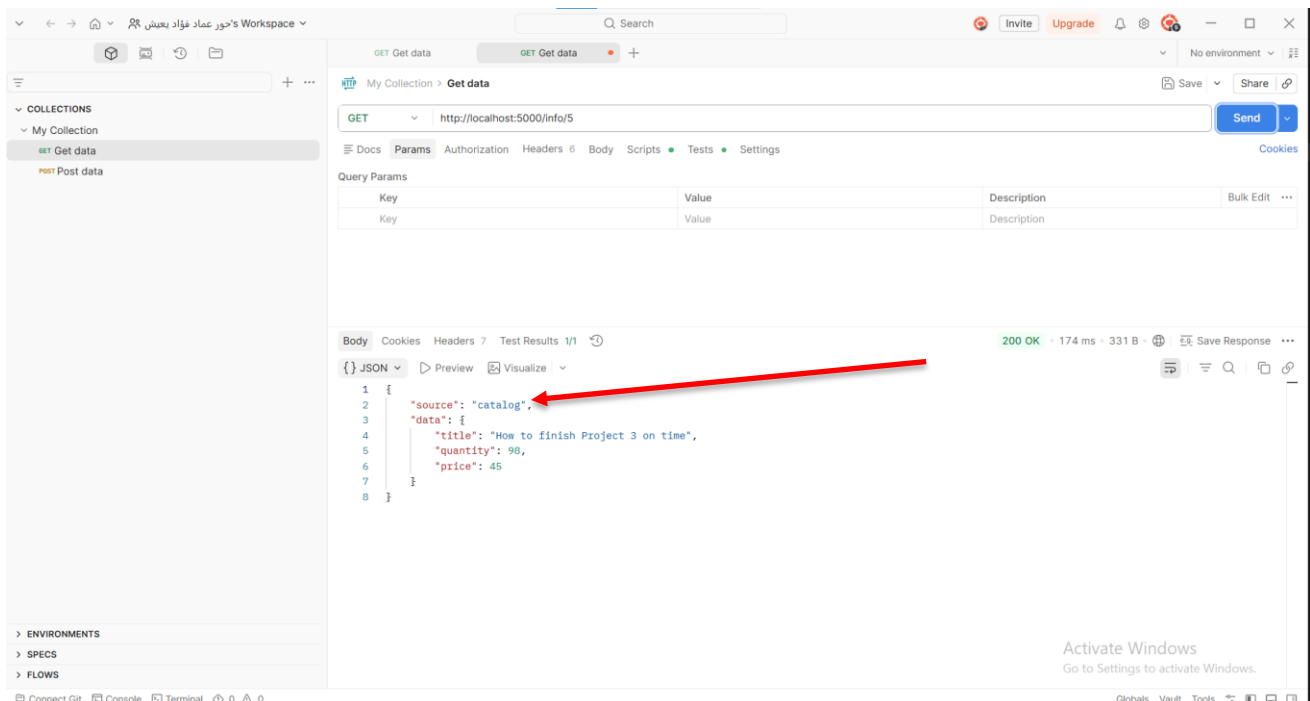


Figure 7: First info request showing source = **catalog**, meaning a cache miss.

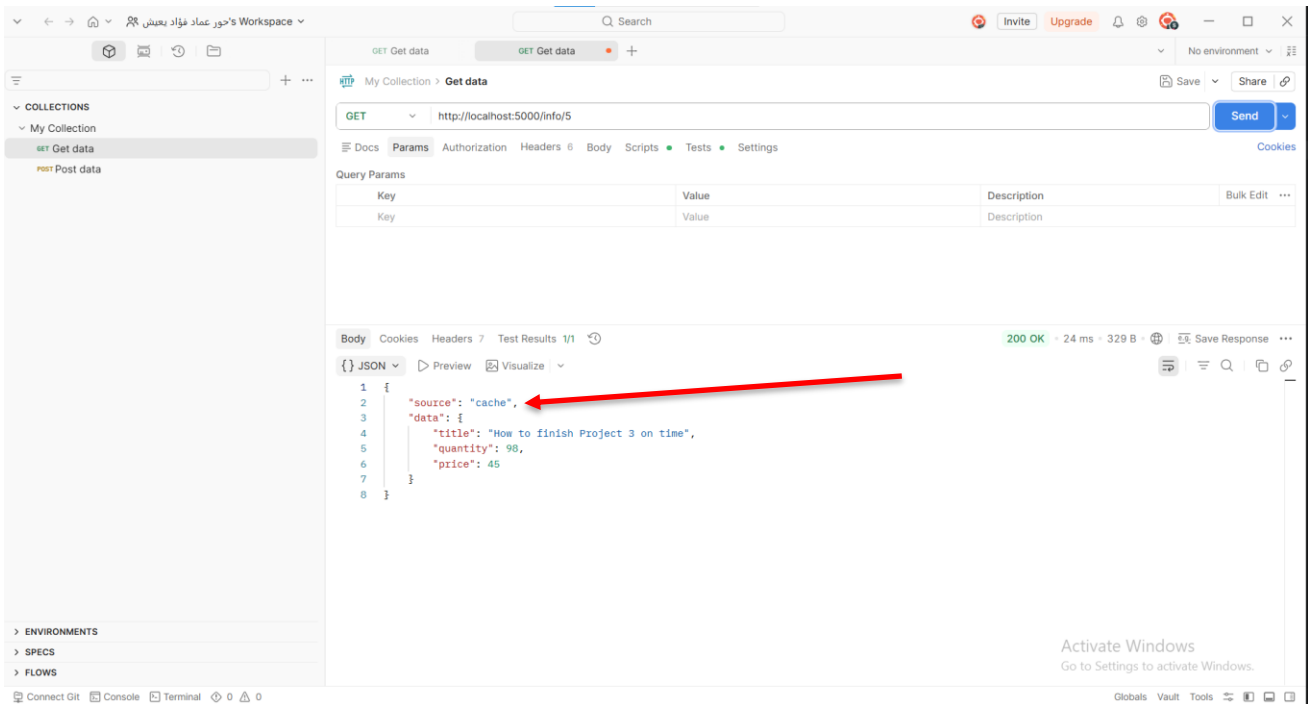


Figure 8: Repeated info request showing source = **cache**, meaning a cache hit.

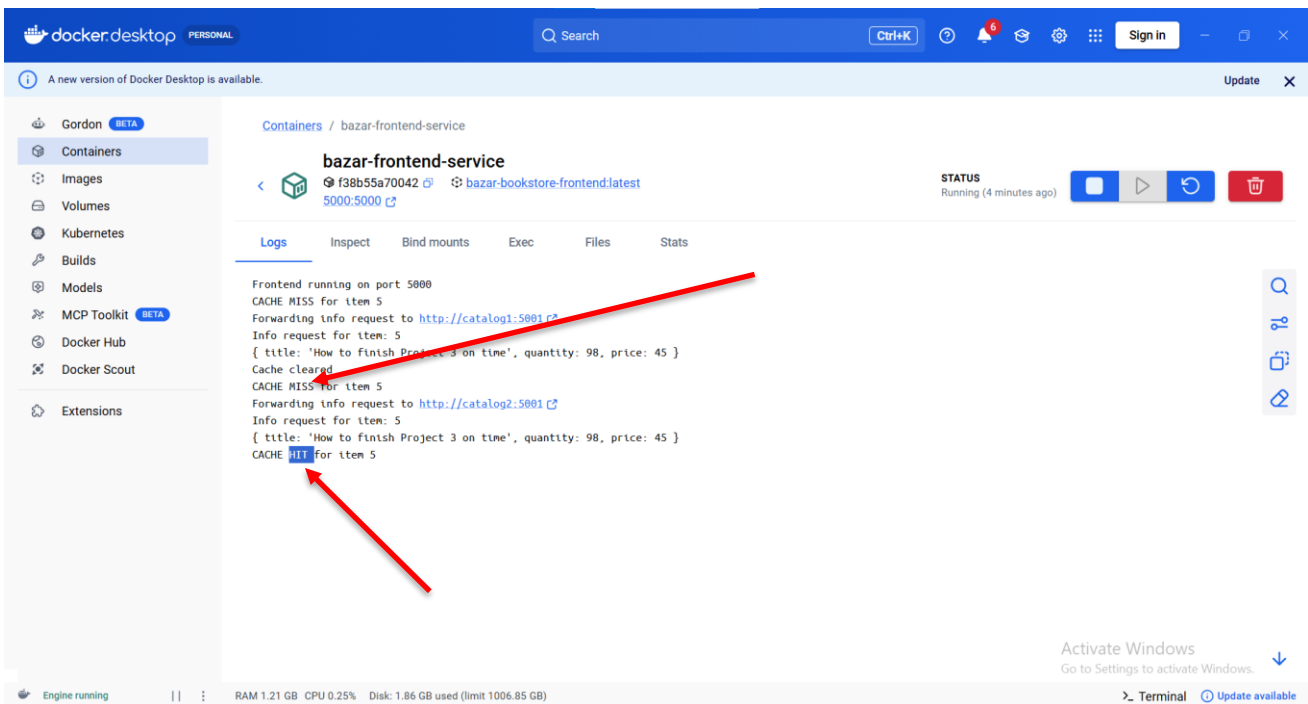


Figure 9: Frontend logs showing **cache miss**, forwarding to catalog, and **cache hit**.

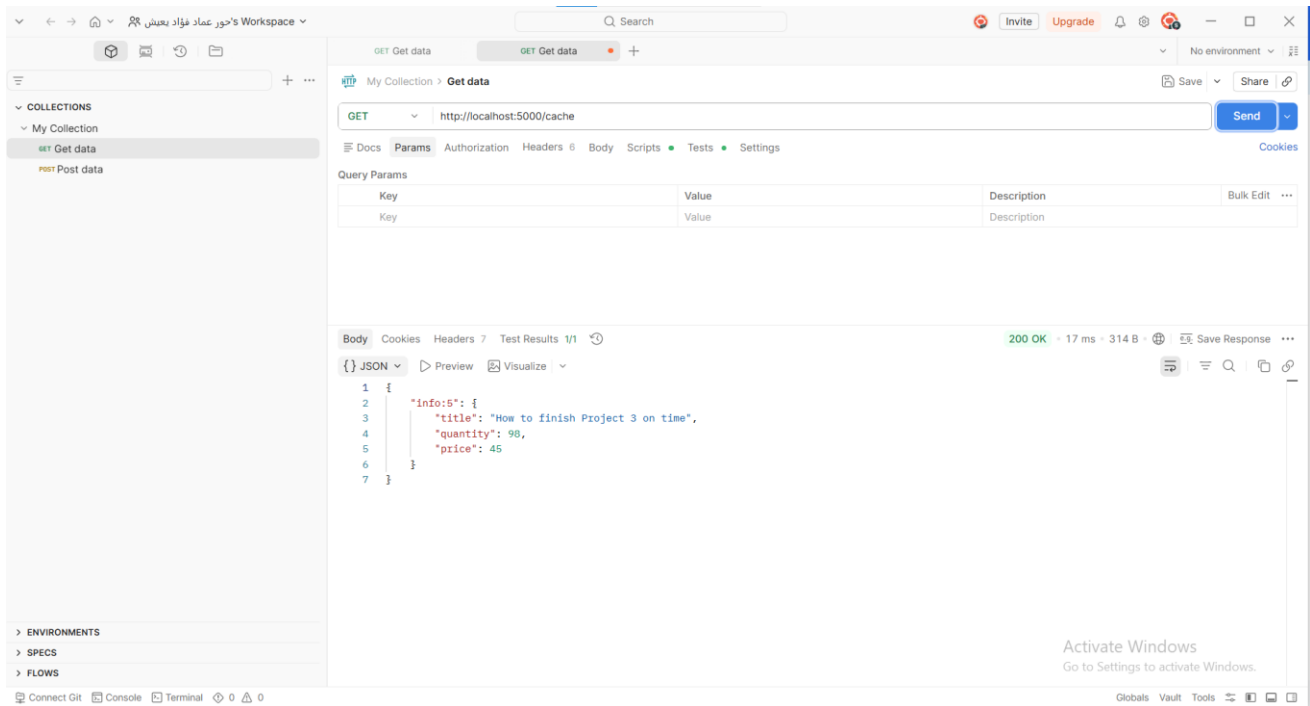


Figure 10: Frontend **cache contents** after an info request.

8. Round-Robin Load Balancing

The frontend uses round-robin load balancing to distribute requests across backend replicas. For catalog requests, the frontend alternates between **catalog1** and **catalog2**. For purchase requests, it alternates between order1 and order2. This avoids sending all requests to one backend service.

Catalog requests: catalog1 -> catalog2 -> catalog1

Order requests: order1 -> order2 -> order1

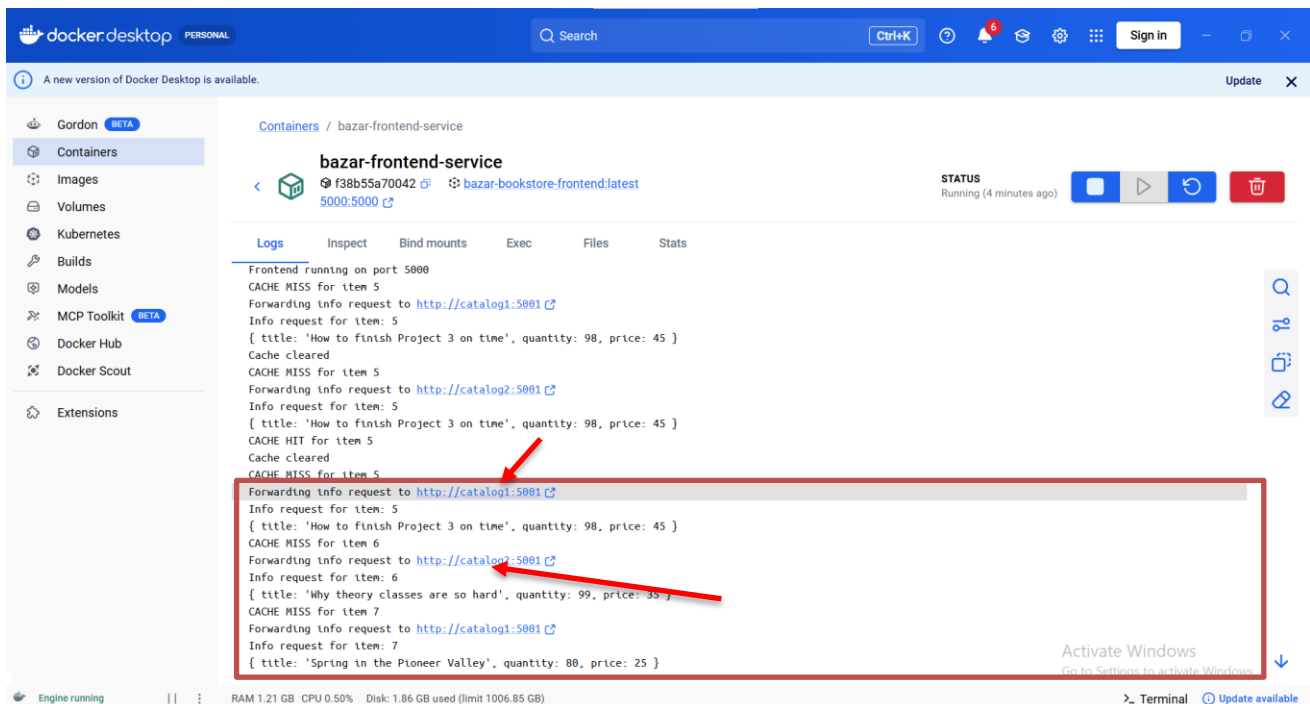


Figure 11: Frontend logs showing round-robin load balancing between catalog replicas.

9. Cache Invalidation and Consistency

Cache invalidation is needed to prevent stale data. For example, if an item is cached with quantity 98 and a purchase reduces the quantity, the cached value should not be returned again. To solve this, the catalog service sends an invalidate request to the frontend before updating the database. The frontend deletes the cached item, so the next info request is forced to retrieve fresh data from the catalog.

POST /invalidate/:id

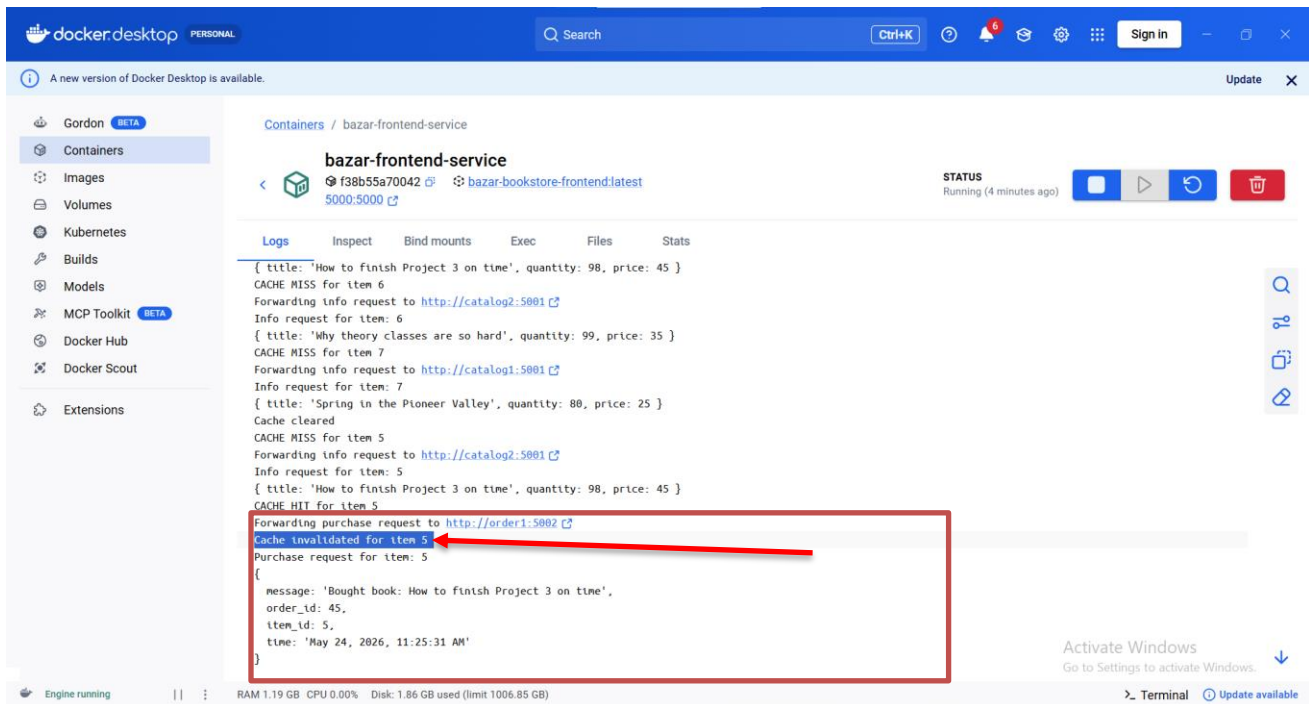


Figure 12: Logs showing cache *invalidation* after a purchase request.

10. Catalog Replica Synchronization

Each catalog replica has its own SQLite database file. When one catalog replica receives an update, it updates its local database and then sends the same update to the other catalog replica using the internal sync endpoint /sync/update/:id. The receiving replica updates its database without forwarding the update again, which avoids an infinite synchronization loop.

PUT /sync/update/:id

After purchasing item 5, both catalog replicas were checked directly. The two replicas returned the same quantity, proving synchronization.

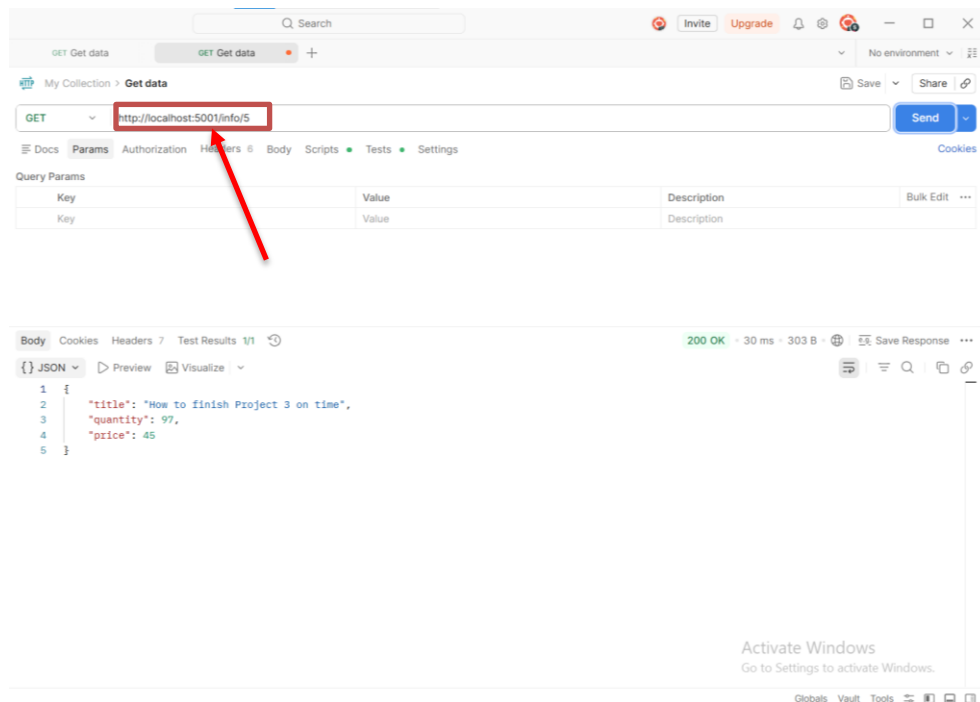


Figure 13: Direct check of catalog *replica 1* after synchronization.

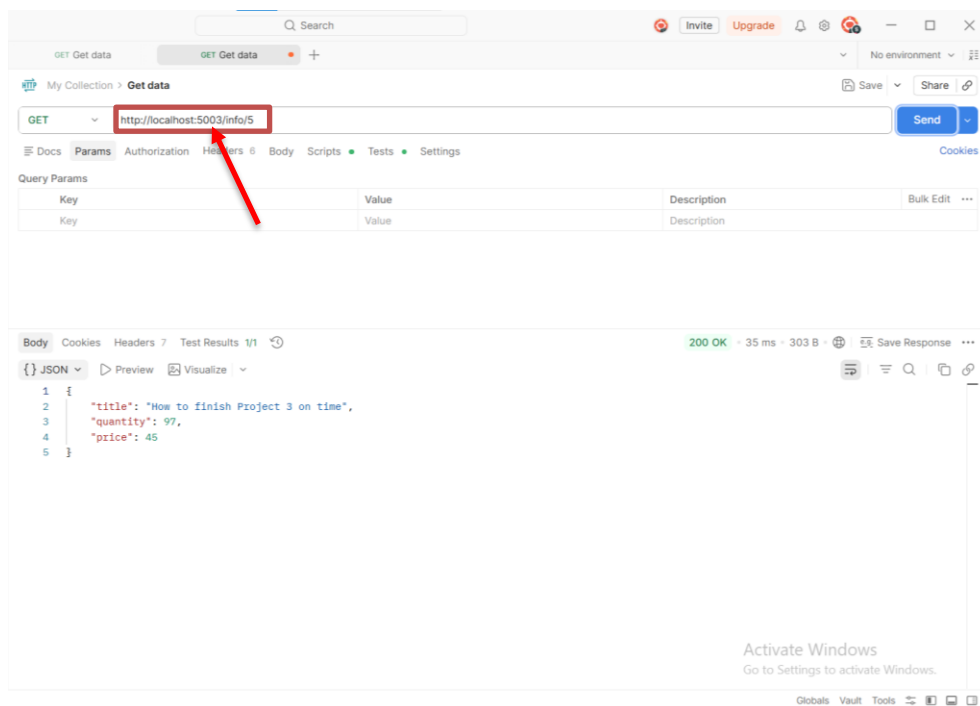


Figure 14: Direct check of catalog *replica 2* showing the same quantity after synchronization.

11. Order Replica Synchronization

The order replicas also synchronize order records. When an order replica receives a purchase request, it stores the order locally and sends the same order record to the other order replica through `/sync/order`. The receiving replica stores the synchronized order but does not send it back again, preventing an infinite loop.

POST /sync/order

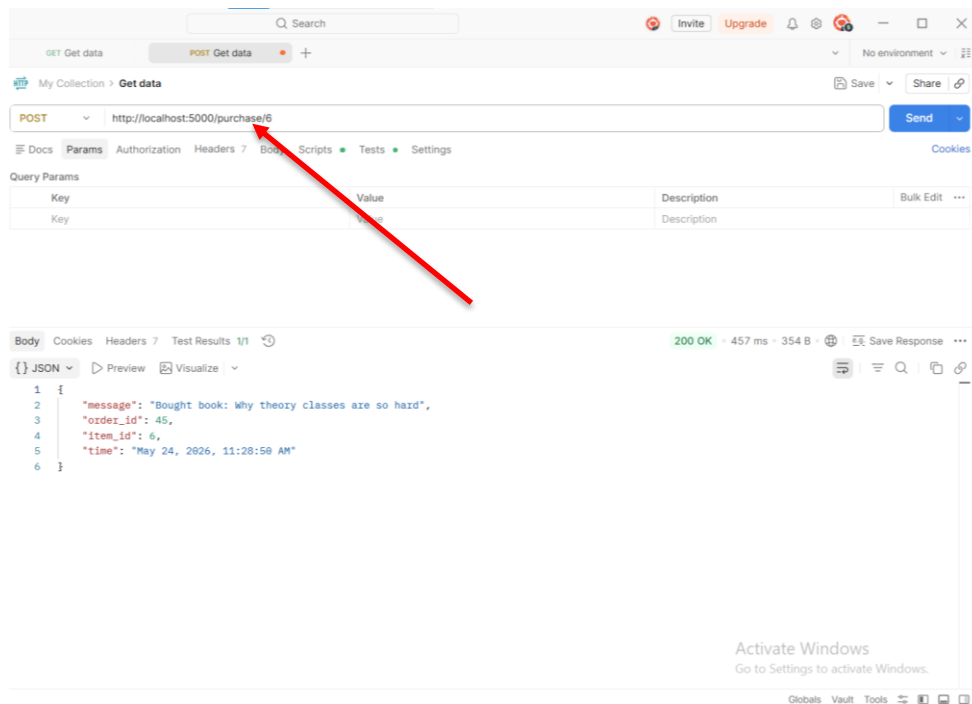


Figure 15: Successful purchase request through the frontend.

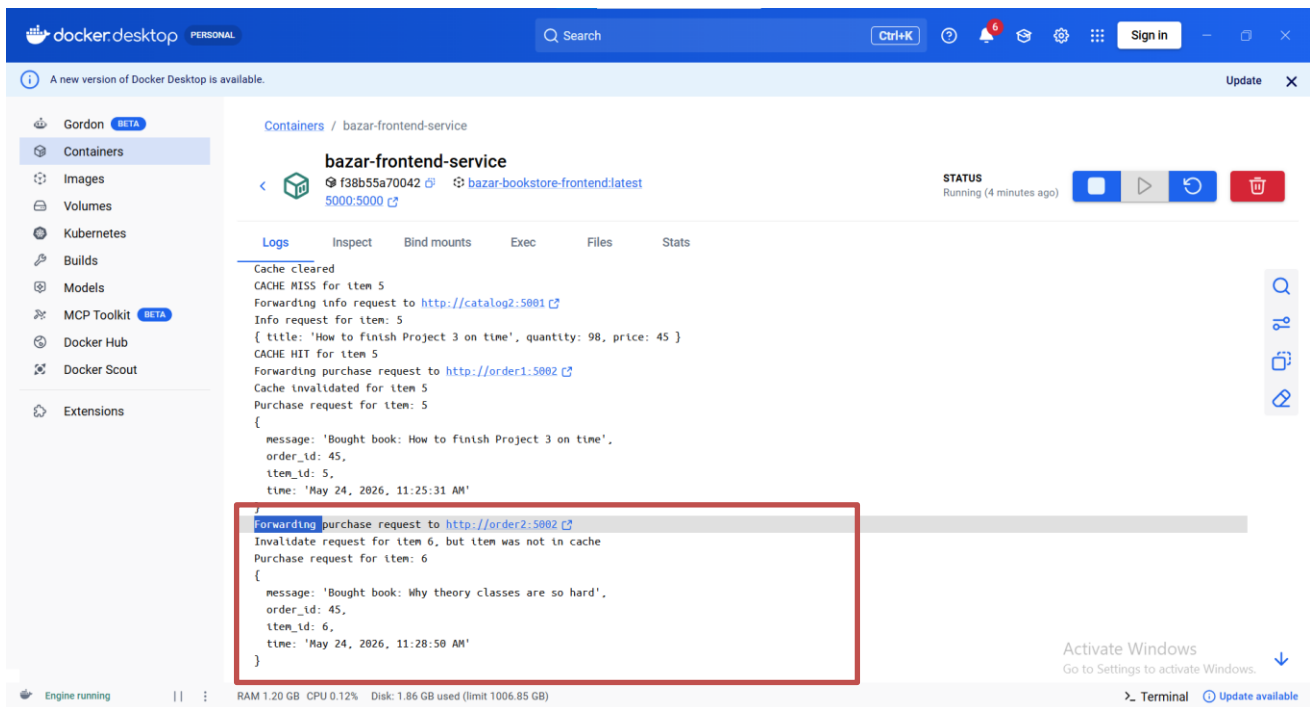


Figure 16: Docker logs showing order synchronization and catalog update synchronization.

The image shows a terminal window with the title 'bazar-bookstore'. The terminal output is divided into three sections, each highlighted with a red box and a red number in a box on the right. Section 1 (labeled '1') shows a purchase request for item 5, an order being bought, and a message containing order_id: 45, item_id: 5, and time: 'May 24, 2026, 11:25:31 AM'. Section 2 (labeled '2') shows a forwarding request to http://order2:5002, an invalidation request for item 6 (not in cache), and a cache invalidation request from bazar-catalog-service-2. Section 3 (labeled '3') shows a synced update for item 6 to replica http://catalog1:5001 and a replicated item 6. Below these sections, there is a purchase request for item 6, followed by an order being bought with order_id: 45, item_id: 6, and time: 'May 24, 2026, 11:28:50 AM'. At the bottom of the terminal, there are links: 'View in Docker Desktop', 'View Config', 'Enable Watch', and 'Detach'.

```
Help  ← → bazar-bookstore  Sign In

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

bazar-frontend-service | Purchase request for item: 5r item 5
bazar-frontend-service | {
bazar-order-service-1  | bought book How to finish Project 3 on time

bazar-frontend-service |   message: 'Bought book: How to finish Project 3 on time',
bazar-frontend-service |   order_id: 45,
bazar-frontend-service |   item_id: 5,
bazar-frontend-service |   time: 'May 24, 2026, 11:25:31 AM'
bazar-frontend-service | }

bazar-frontend-service | Forwarding purchase request to http://order2:5002
bazar-frontend-service | Invalidate request for item 6, but item was not in cache
bazar-catalog-service-2 | Sent cache invalidation request for item 6

bazar-catalog-service-2 | Synced update for item 6 to replica http://catalog1:5001
bazar-catalog-service-1 | Replica synchronized item 6

bazar-frontend-service | Purchase request for item: 6

bazar-order-service-2  | bought book Why theory classes are so hard',
bazar-frontend-service |   order_id: 45,
bazar-frontend-service |   item_id: 6,
bazar-frontend-service |   time: 'May 24, 2026, 11:28:50 AM'
bazar-frontend-service | }

View in Docker Desktop  View Config  Enable Watch  Detach
```

Figure 17: Terminal logs showing **purchase processing**, **cache invalidation**, **catalog synchronization**, and **order synchronization**.

12. Performance Measurements

A performance experiment script was created in `experiments/measure.js`. The script measures average response time for info requests without cache, info requests with cache, purchase requests, cache invalidation writes, and cache miss after invalidation. During the experiment, quantities were temporarily increased to avoid out-of-stock errors during repeated purchases.

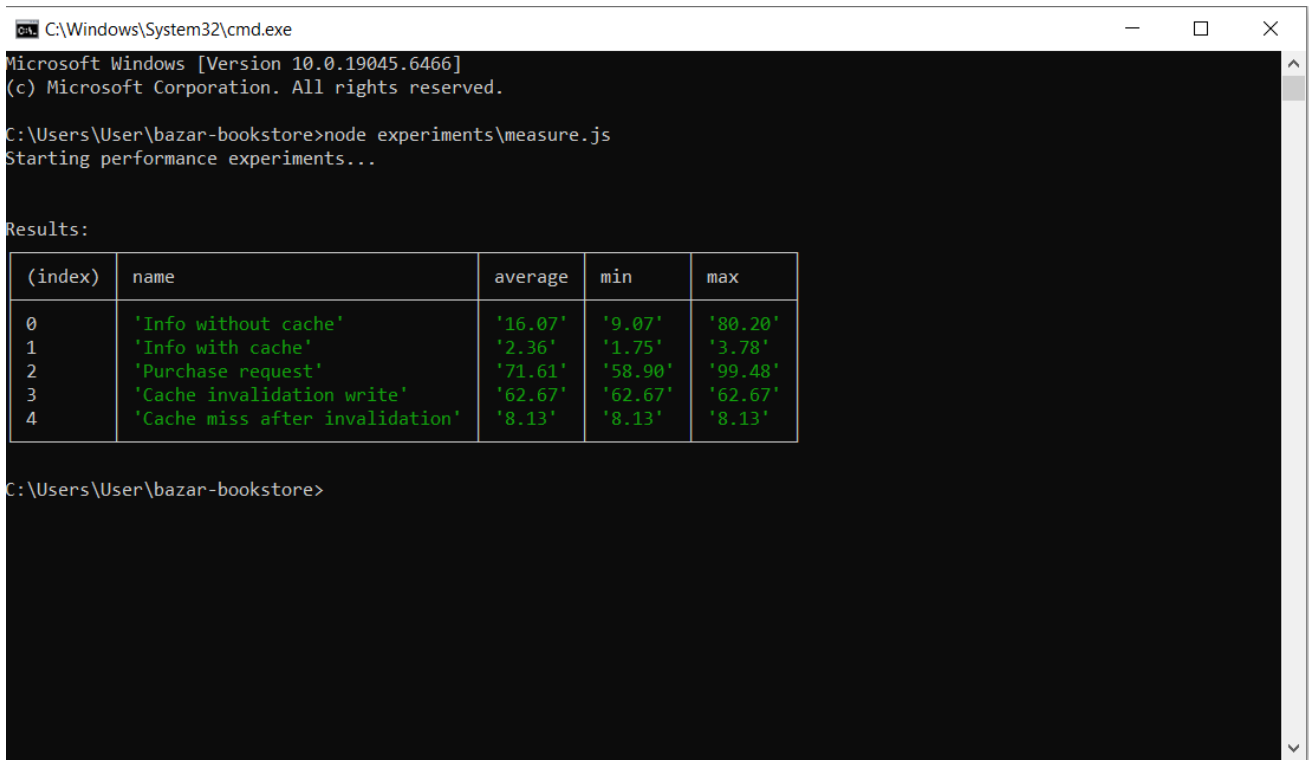


Figure 18: Performance measurement results comparing response times with and without caching.

Experiment	Average (ms)	Minimum (ms)	Maximum (ms)
Info without cache	16.07	9.07	80.20
Info with cache	2.36	1.75	3.78
Purchase request	71.61	58.90	99.48
Cache invalidation write	62.67	62.67	62.67
Cache miss after invalidation	8.13	8.13	8.13

The results show that caching significantly improves read performance. The average info request without cache was 16.07 ms, while the average info request with cache was 2.36 ms. Purchase requests are slower because they involve the order service, catalog update, database write, cache invalidation, and replica synchronization.

13. Design Tradeoffs

The cache was implemented inside the frontend instead of as a separate cache service. This made the implementation simpler and reduced the number of REST calls. The tradeoff is that the cache is lost when the frontend restarts.

Round-robin load balancing was selected because it is simple and easy to verify using logs. A more advanced least-loaded strategy could distribute requests based on actual server load, but it would require additional monitoring.

SQLite was used for persistence because it is lightweight and suitable for this lab. For a larger real-world system, a more scalable database system or a message queue could be used.

14. Known Limitations and Possible Improvements

Although the system satisfies the Lab 2 requirements, it still has limitations. The frontend is not replicated, so it remains a single point of failure. Synchronization is implemented using simple REST calls, so if a replica is down during synchronization, the update may fail and there is no automatic retry mechanism. Also, the system does not implement a consensus protocol for strong consistency.

Possible improvements include adding retry logic, health checks, frontend replication, least-loaded load balancing, cache size limits, LRU cache replacement, structured logging, and automated tests.

15. How to Run the Project

To run the Lab 2 system, open a terminal inside the project folder and execute:

```
docker compose down --remove-orphans
docker compose up --build
```

The main frontend URL is:

```
http://localhost:5000
```

Example API requests:

```
GET    http://localhost:5000/info/5
GET    http://localhost:5000/cache
DELETE http://localhost:5000/cache
POST   http://localhost:5000/purchase/5
GET    http://localhost:5001/info/5
GET    http://localhost:5003/info/5
```

To run the performance experiment:

```
node experiments\measure.js
```

16. Conclusion

Part 2 successfully extended the Bazar Bookstore project from a basic multi-tier microservice application into a more distributed system. The final implementation includes backend replication, frontend caching, cache invalidation, replica synchronization, round-robin load balancing, Docker deployment, and performance measurements. The experiment results show that caching reduces read request latency significantly, while synchronization and invalidation maintain consistency after writes.